TRIE INDEXING — CLASS LECTURE NOTES

1. What is a Trie?

A Trie (pronounced “try”) is a tree-based data structure used to store and search strings efficiently. Each node usually represents a character, and a path from the root represents a word or prefix.

2. What is a Prefix?

A **prefix** is a sequence of characters that appears at the beginning of a word.

Examples:

CAT → C, CA, CAT are prefixes.
CAR → C, CA, CAR are prefixes.
COMPUTER → C, CO, COM, COMP, ... are prefixes.

Important: In a Trie, words having the same prefix share the same path.

3. Simple Trie Example

Store the words CAT, CAR and CAN:

```
ROOT
■
C
■
A ← common prefix “CA”
/ ■ \
T R N
* * *
```

4. Trie Indexing

An index helps us find data quickly instead of scanning every item. A Trie can act as an index for strings by organizing them character by character.

Example words: apple, application, apply, banana, ball, bat.

If we search for the prefix **“app”**, we follow ROOT → A → P → P and can find words beginning with that prefix.

5. Main Operations

Insertion: Add a word character by character and mark the final node as the end of the word.

Searching: Follow the characters from the root. If all characters exist and the final node is marked as an end of a word, the word exists.

Prefix Search: Follow the prefix and then explore the nodes below it to find all matching words.

6. Prefix Search Example

If the Trie contains CAR, CARD, CARE, CART and CAT, searching for prefix “CAR” can return:

```
CAR
CARD
CARE
CART
```

7. Time Complexity

Operation	Time Complexity
Insertion	O(L)
Search	O(L)
Prefix Search	O(L + K)

Here, L is the length of the string and K represents the amount of result data that must be visited/output.

8. Real-World Applications

• Search suggestions and autocomplete
• Dictionaries
• Spell checking
• Prefix-based searching
• IP routing and other indexing applications

9. Advantages and Disadvantages

Advantages: Fast string searching, excellent prefix matching, and shared storage of common prefixes.

Disadvantage: Tries can use considerable memory because nodes may contain links for many possible characters.

10. Short Conclusion for Seminar

A Trie is a tree-based indexing structure designed for strings. It stores characters along paths and shares common prefixes. This makes searching and prefix matching efficient. Tries are especially useful in autocomplete, dictionaries, spell checkers and search systems.